

Week 2: Streaming, Sketching, and Probabilistic Counting

DSAA1001 · Three-tier learning sheet · Advanced Learning Sheet

Opening diagnostic challenge. The campus network team sees millions of packet records per hour. They want to know which services are popular, which IPs are suspicious, how many flows have appeared, and whether a destination is becoming unusually common. They **cannot store all packet records** and cannot go back to reread yesterday's stream.

Before choosing an algorithm, what must they decide about memory, accuracy, failure probability, and whether false positives or false negatives are acceptable?

Reading contract and capabilities

Week orientation

Backbone question. How can we answer useful questions about a fast stream when exact storage is impossible, by **choosing a compact state and calculating the error it introduces?**

Running example. A campus network operations team observes packet and flow records of the form (timestamp, source_ip, destination_ip, protocol, service, flow_id, packet_size, user_id). They want to count traffic, preserve a sample for inspection, test whether a suspicious IP or URL has appeared, and estimate service frequencies (with heavy-hitter triage discussed in Appendix B).

A **streaming algorithm** will mean an algorithm that **reads each item once**, maintains compact state, and answers a query from that state.

Learning objectives

- explain the data stream model;
- distinguish exact, approximate, randomized, and high-probability answers;
- prove reservoir sampling uniformity;
- analyze simple probabilistic counters and Morris counting;
- explain Bloom Filter one-sided error;
- analyze Count-Min Sketch overestimation and amplification;
- use sketch estimates for heavy-hitter decisions;
- critique whether an algorithmic guarantee matches an operational risk.

Breadcrumb Prompt

Write your starting rule of thumb

In one sentence, write when you think approximation is acceptable for a campus network monitoring question. At the end of the sheet, revise it into a rule that names the query, memory limit, error type, and action risk.

- **Tier 1: operational intuition and immediate use.** What the campus team can do with the method.
- **Tier 2: formal definitions, algorithms, and proof-guided claims.** Why the guarantee holds under the assumptions.
- **Tier 3: calculation and judgment.** Use the guarantees on a concrete stream, then decide whether the error is acceptable.

Remark. Notation map. Use t for stream time in process descriptions, n for a fixed true count in counting examples, and N for total stream mass in frequency sketches. C_t is a random counter after t items; lowercase c is **one possible observed value**. f_x is the true frequency of key x . $\hat{f}_x$ will mean a one-row frequency estimate, while $\hat{f}_x$ means a final estimator. In sketches, w is width, d is depth, ε is an accuracy parameter, and δ is a failure-probability parameter.

1. Data streams and compact state

The campus network question is: how can the team monitor packet records as they arrive, without storing all past records and without rereading old traffic? The naive method is to append every record to a database and run exact queries later. That fails when the stream is too fast, too large, privacy-sensitive, or only available once. The compact-state trick is to keep only a small memory summary that is updated per item. The price is that the summary must be **designed for the query**: it may lose history, approximate answers, or hide bursts that matter operationally.

Tier 1: one pass, small memory

A stream is a sequence of items that arrives too quickly, too indefinitely, or too expensively to store in full. A streaming algorithm processes each packet or flow record once, updates a small state, and can answer a query using only that state.

For the campus network stream, different questions suggest different compact states:

- count packets or flows so far: a counter or probabilistic counter;
- keep examples for inspection: a sample reservoir;
- ask whether a URL was seen: a bit-array membership structure;
- estimate frequencies: a sketch of hashed counters;
- triage unusually common destinations: candidate keys plus approximate counts.

Tier 2: formal stream model

Definition 1: A data stream is a sequence $S = (a_1, a_2, \dots, a_t, \dots)$ of items from a **universe** U (the set of all possible items that could appear, such as all possible IP addresses or all possible service names). A streaming algorithm maintains memory state

$$M_t = \text{update}(M_{t-1}, a_t)$$

and answers a query by returning $\text{query}(M_t)$.

Traditional data structures fail here because storing exact counts for every possible key (every IP address, every service name, every flow ID) requires memory proportional to the universe size—infeasible when the universe contains billions of potential keys. Even if we could afford the memory, exact sorting and enumeration become too slow when decisions must be made in real time.

A **sketch** is a compact summary structure that supports four operations: create an empty summary, update it with one new tuple, merge two summaries when data arrives from distributed monitors, and query the summary. Sketches can be viewed as sufficient statistics for streaming: they **preserve enough information for the intended queries**, but not the whole dataset. A **mergeable** sketch means two summaries can be **combined without replaying raw data**—essential when data arrives from multiple routers or time windows.

The stream model has five practical axioms:

- **one pass**: read each item as it arrives;
- **low update time**: process and discard quickly;
- **sublinear memory**: keep summaries or sketches rather than raw history;
- **anytime answers**: answer queries from current state;
- **evolving stream**: the distribution or operational regime may change over time.

For the campus network, U may contain all possible tuples (timestamp, source_ip, destination_ip, protocol, service, flow_id, packet_size, user_id), or it may contain derived keys such as destination_ip, service, or flow_id, **depending on the query**.

Definition 2: In both models, let A be a **frequency vector** (an array) indexed by keys, so $A[j]$ is the current count or weight of key j . For example, $A[\text{HTTP}]$ might count the number of HTTP packets seen so far.

In the **cash-register model** (also called arrivals-only), stream update (j, I) changes the frequency vector by $A[j] \leftarrow A[j] + I$ with $I \geq 0$. The name “cash register” comes from retail checkout: items are only added, never removed. Packet counts by source IP or destination service fit this model.

In the **turnstile model** (named after subway entry/exit gates that can increment or decrement counts), updates may be positive or negative: $A[j] \leftarrow A[j] + U$. Active-flow counts fit this model: a flow start adds 1 and a flow end subtracts 1.

Definition 3: The **space** of a streaming algorithm is the number of bits or machine words in M_t . The **update time** is the time to process one new item. The **query time** is the time to answer from M_t . A mergeable sketch also supports combining summaries from different monitors without replaying raw streams.

The update and query model lets us compare compact states by calculating exactly what each state retains.

Tier 3: the same stream, two compact states

For the service stream

(HTTP, DNS, HTTP, SSH, HTTP, DNS),

consider two algorithms. Algorithm A stores only the total count $M_t = t$. Algorithm B stores a frequency vector for the services that have appeared. After six updates,

$$M_6 = 6, \quad A[\text{HTTP}] = 3, \quad A[\text{DNS}] = 2, \quad A[\text{SSH}] = 1.$$

Both states answer the total-count query. Only the frequency vector answers how many HTTP records appeared. The state must therefore be chosen for the intended query.

Key calculation: State follows query

Applying $M_t = \text{update}(M_{t-1}, a_t)$ six times gives a one-number state for total count and a three-counter state for per-service counts. The additional two counters preserve the information needed for the more detailed query.

Practice: Stream state calculation

Trace the stream (DNS, HTTP, DNS, Video, DNS, HTTP, DNS, SSH). Compute the final state for (a) a total counter and (b) a frequency vector. Answer the total-count and DNS-count queries from each state, and state which query the total counter cannot answer.

Capability Checkpoint: Stream model and compact state Work questions Q1–Q4 in the end-of-sheet **Checkpoint Question Bank**.

2. Approximation, randomness, and failure probability

The campus network question is: can the dashboard answer quickly when exact answers are too costly? The naive method is to demand exact counts, exact membership, and exact rankings for every key. That fails when the key universe is huge and traffic never stops. The compact randomized trick is to **accept a stated error budget: an estimator may be unbiased, one-sided, or correct with high probability**. The price is that “accurate” now has parameters, usually an accuracy knob ε and a failure-probability knob δ .

Tier 1: randomness as an algorithm resource

Randomness is not only noise in the data. It can be a **design resource used by the algorithm**. Two runs on the same packet stream may keep different samples or create different hash collisions, but the algorithm is designed so that most runs answer within a useful error range.

Operationally, the campus team must separate four statements:

- **exact**: the answer is the true value;
- **approximate**: the answer is close enough by a specified tolerance;
- **randomized**: the answer depends on random bits or hash choices;
- **high-probability**: the **chance of violating the error tolerance** is at most a specified δ .

Tier 2: accuracy, confidence, and variance reduction techniques

Definition 4: An estimator $\hat{\theta}$ has **absolute error at most ε** with failure probability δ if

$$\Pr(|\hat{\theta} - \theta| > \varepsilon) \leq \delta.$$

It has **relative error at most ε** if

$$\Pr(|\hat{\theta} - \theta| > \varepsilon\theta) \leq \delta$$

when $\theta > 0$.

How do we achieve (ε, δ) guarantees in practice? Consider an unbiased estimator $\hat{\theta}$ with $E[\hat{\theta}] = \theta$. Even though the expectation is correct, **a single sample $\hat{\theta}$ could be very far from θ** . If individual samples are unreliable, extracting useful information from **one** sample is hopeless, and even using **several** independent samples may not help without additional structure.

The key question: **Can we bound the probability that $\hat{\theta}$ deviates from θ by more than ε ?** This probability is called the **tail probability**. If we have additional information about $\hat{\theta}$ —such as its variance, higher moments, or boundedness—we can quantify tail probabilities using **concentration inequalities**.

Concentration inequalities: Bounds such as Markov, Chebyshev, Chernoff, or Hoeffding quantify how tightly a random variable concentrates around its mean. **Each inequality uses different information:**

- **Markov:** uses only $E[X]$ (weakest, applies to non-negative random variables)
- **Chebyshev:** uses $E[X]$ and $\text{Var}(X)$ (no independence needed)
- **Chernoff/Hoeffding:** use independence of summands and boundedness (strongest tail bounds)

For example, Chebyshev’s inequality states that for any random variable X with mean μ and variance σ^2 ,

$$\Pr(|X - \mu| \geq \varepsilon) \leq \frac{\sigma^2}{\varepsilon^2}.$$

This converts moment information (mean, variance) into a tail probability guarantee.

Boosting/amplification: Suppose we have an estimator that succeeds with only constant probability, say $\Pr(|\hat{\theta} - \theta| \leq \varepsilon) \geq \frac{2}{3}$. How do we boost this to *arbitrarily* high probability $\geq 1 - \delta$?

The trick: run k **independent trials and take the median**. If trials are independent, the probability that **more than half** fail can be bounded by a Chernoff bound. Specifically, let B be the number of failed trials. Then $B \simeq \text{Binomial}(k, \frac{1}{3})$ with $E[B] = \frac{k}{3}$. **The median fails only if $B \geq \frac{k}{2}$** . By Chernoff,

$$\Pr\left(B \geq \frac{k}{2}\right) \leq \exp(-\Theta(k)).$$

Choosing $k = \Theta(\ln(\frac{1}{\delta}))$ makes this $\leq \delta$. Thus $O(\ln(\frac{1}{\delta}))$ independent trials suffice to boost constant success probability to high probability.

Now we can build variance reduction techniques on these foundations:

Theorem 1: Variance reduction by averaging. Let $X_1, \dots, X_r$ be independent unbiased estimators of θ with common variance σ^2 . Their average $\bar{X} = (\frac{1}{r}) \sum_{j=1}^r X_j$ satisfies

$$E[\bar{X}] = \theta, \quad \text{Var}(\bar{X}) = \frac{\sigma^2}{r}.$$

Proof: Unbiasedness: $E[\bar{X}] = (\frac{1}{r}) \sum_{j=1}^r E[X_j] = (\frac{1}{r}) \cdot r\theta = \theta$. For variance,

$$\text{Var}(\bar{X}) = \text{Var}\left(\left(\frac{1}{r}\right) \sum_{j=1}^r X_j\right) = \left(\frac{1}{r^2}\right) \sum_{j=1}^r \text{Var}(X_j) = \left(\frac{1}{r^2}\right) \cdot r\sigma^2 = \frac{\sigma^2}{r},$$

using independence (so cross-terms vanish) and identical variance. $\square$

Remark. Role of independence. The variance formula $\text{Var}(\bar{X}) = \frac{\sigma^2}{r}$ holds only when $X_1, \dots, X_r$ are independent. If the estimators are positively correlated, **cross-terms $\text{Cov}(X_i, X_j)$ do not vanish, and variance reduction is weaker or absent**. For example, if $X_i = X_1$ for all i (perfect correlation), then $\text{Var}(\bar{X}) = \sigma^2$ —no reduction at all despite averaging r copies.

Independence is crucial. If trials are correlated, one failure can cause many failures, and the median offers no protection. The exponential tail bound $\exp(-\Theta(k))$ relies on independence. Without it, we would need polynomial $O(\frac{1}{\delta})$ trials instead of logarithmic $O(\ln(\frac{1}{\delta}))$.

This reduces variance but **does not eliminate outliers**: one very large X_j can still pull $\bar{X}$ far from θ .

Theorem 2: Median-of-means probability amplification. Divide $r = g \cdot s$ independent estimators into g groups of size s each. Let G_i be the average within group i , and return $\text{median}(G_1, \dots, G_g)$.

If each X_j is an unbiased estimator with variance σ^2 , and $s = \Theta\left(\frac{\sigma^2}{\varepsilon^2}\right)$, then by Chebyshev each group mean satisfies

$$\Pr(|G_i - \theta| \leq \varepsilon) \geq \frac{2}{3}.$$

If $g = \Theta\left(\ln\left(\frac{1}{\delta}\right)\right)$ independent groups each succeed with probability $\geq \frac{2}{3}$, then the median is ε -close to θ with probability $\geq 1 - \delta$ by a Chernoff bound on the number of bad groups.

Total sample complexity: $r = gs = O\left(\left(\frac{\sigma^2}{\varepsilon^2}\right) \cdot \ln\left(\frac{1}{\delta}\right)\right)$.

Proof: Within-group variance reduction: $\text{Var}(G_i) = \frac{\sigma^2}{s}$. By Chebyshev,

$$\Pr(|G_i - \theta| > \varepsilon) \leq \frac{\text{Var}(G_i)}{\varepsilon^2} = \frac{\sigma^2}{s\varepsilon^2}.$$

Choose $s = 3\frac{\sigma^2}{\varepsilon^2}$ so $\Pr(|G_i - \theta| \leq \varepsilon) \geq \frac{2}{3}$.

Let B be the number of bad groups (where $|G_i - \theta| > \varepsilon$). Then $E[B] = \frac{g}{3}$. The median is bad only if $B \geq \frac{g}{2}$. By a Chernoff bound on $B \simeq \text{Binomial}(g, \frac{1}{3})$,

$$\Pr\left(B \geq \frac{g}{2}\right) \leq \exp(-\Theta(g)).$$

Choose $g = \Theta\left(\ln\left(\frac{1}{\delta}\right)\right)$ to make this $\leq \delta$. □

Remark. Two roles of independence. First, independence within each group allows variance reduction: $\text{Var}(G_i) = \frac{\sigma^2}{s}$. Second, **independence across groups allows boosting**: the probability that many independent 2/3-success events all fail is exponentially small. If groups are correlated, one bad group could make many groups bad, destroying the median's protection.

These theorems appear throughout Week 2: reservoir sampling uses unbiased estimators, Morris counters apply variance reduction by averaging, and Count-Min Sketch uses median-of-means for probability amplification.

Tier 2 gives both unbiasedness and concentration bounds. Use the two guarantees on one numerical estimator before deciding whether it is precise enough.

Tier 3: from moments to an error probability

Suppose an unbiased estimator has true value $\theta = 100$, variance $\sigma^2 = 36$, and we average $r = 4$ independent copies. Tier 2 gives

$$E[\bar{X}] = 100, \quad \text{Var}(\bar{X}) = \frac{36}{4} = 9.$$

For an absolute error tolerance of $\varepsilon = 10$, Chebyshev gives

$$\Pr(|\bar{X} - 100| \geq 10) \leq \frac{9}{100} = 0.09.$$

The expectation says the average is correct; the variance and bound quantify the chance of a large error.

Key calculation: Expectation versus guarantee

An unbiased estimate can still miss by 10 or more. Here averaging four copies changes the variance from 36 to 9 and gives a failure bound of 0.09 for the chosen tolerance.

Practice: Calculate an (ε, δ) guarantee

A dashboard averages $r = 9$ independent estimators with variance 81 for a target count $\theta = 500$. Compute the variance of the average and the Chebyshev upper bound for an absolute error of $\varepsilon = 12$. Then state the corresponding failure probability δ and whether a requested $\delta = 0.05$ is certified by this bound.

Capability Checkpoint: Approximation and randomized guarantees Work questions Q5–Q8 in the end-of-sheet **Checkpoint Question Bank**.

3. Reservoir sampling: a uniform sample without knowing the stream length

The campus network question is: can the team preserve 100 packet records for later manual inspection without favoring the beginning or the end of the day? The naive method is to store every packet and choose a sample after the stream ends. That fails because the stream length is unknown and storage is too large. The trick is to maintain a reservoir of k records, replacing old records with **carefully chosen probabilities** as new records arrive. The price is randomness: any one run may omit an interesting packet, but **every packet position is equally likely to appear in the final reservoir**.

Tier 1: keep a fair sample while the stream grows

Reservoir sampling keeps a uniform sample of size k from a stream whose **final length is unknown**. In the campus network, the team might keep $k = 100$ packet records for later inspection while millions of records pass by.

A simpler velocity-control idea is Bernoulli sampling: process item s_t with probability α , discard it otherwise, and extrapolate from the sampled items. This reduces processing load but **does not guarantee a fixed memory budget**. Reservoir sampling solves the memory version: **keep exactly k records** while preserving equal inclusion probability.

Algorithm for item s_t :

1. Store the first k items.
2. For $t > k$, keep s_t with probability $\frac{k}{t}$.
3. If kept, replace one existing reservoir item chosen uniformly at random.

Tier 2: proof-guided uniformity claim

Theorem 3: After processing $t \geq k$ stream items, reservoir sampling satisfies

$$\Pr(s_i \text{ is in the reservoir after time } t) = \frac{k}{t}$$

for every $i \leq t$.

Proof: Use induction on t . At $t = k$, all first k items are stored, so each has probability $1 = \frac{k}{k}$.

Assume after time t every old item s_i , $i \leq t$, is in the reservoir with probability $\frac{k}{t}$. At time $t + 1$, the new item is accepted with probability $\frac{k}{t+1}$. If it is accepted, it replaces a particular old reservoir slot with probability $\frac{1}{k}$. Thus an old item currently in the reservoir **is removed with probability**

$$\left(\frac{k}{t+1}\right) \cdot \left(\frac{1}{k}\right) = \frac{1}{t+1}.$$

Therefore, for $i \leq t$,

$$\Pr(s_i \in S_{t+1}) = \left(\frac{k}{t}\right) \cdot \left(1 - \frac{1}{t+1}\right) = \left(\frac{k}{t}\right) \cdot \left(\frac{t}{t+1}\right) = \frac{k}{t+1}.$$

The new item s_{t+1} is inserted with probability $\frac{k}{t+1}$. Hence every item among the first $t + 1$ items has inclusion probability $\frac{k}{t+1}$. $\square$

Tier 2 gives the inclusion probability $\frac{k}{t}$. Use it to compare a whole-history sample with a recent-window requirement.

The proof is about **inclusion probability of each item**. Stronger statements about all k -subsets being equally likely also hold for the standard algorithm, but the inclusion guarantee is the key operational fact for unbiased inspection.

Tier 3: calculate what the reservoir represents

With $k = 2$ and $t = 5$, each of the five stream positions has inclusion probability $\frac{2}{5}$. For an early item s_1 , the survival product is

$$1 \cdot \left(1 - \frac{1}{3}\right) \cdot \left(1 - \frac{1}{4}\right) \cdot \left(1 - \frac{1}{5}\right) = \frac{2}{5}.$$

For a recent-window question, suppose the last two items are the only attack records. The expected number of attack records in the reservoir is

$$2 \cdot \left(\frac{2}{5}\right) = 0.8,$$

even though a uniform sample remains correct for whole-history inspection. The **sample design must therefore match the query window**.

Key calculation: Inclusion probability

Reservoir sampling gives every position probability $\frac{k}{t}$, not extra weight for recent positions. With $k = 2$ and $t = 5$, that probability is 0.4 for every item.

Practice: Calculate reservoir inclusion

For $k = 4$ and $t = 20$, compute (a) the probability that a new item is accepted, (b) the probability that a particular old reservoir item is replaced at that step, and (c) the expected number of items from the last five positions in the final reservoir. State whether the last-five expectation answers a recent-attack query reliably.

Capability Checkpoint: Reservoir sampling Work questions Q9–Q12 in the end-of-sheet **Checkpoint Question Bank**.

4. Probabilistic counting and Morris counter

The campus network question is: how can a tiny device or per-flow monitor estimate how many packets have arrived? The naive method is an exact binary counter, which needs about $\log_2 t$ bits to count up to t . That is already small for one counter, but expensive when repeated across many monitored keys or very small devices. The randomized trick is to increment only sometimes and let the counter value **represent an exponential scale**. The price is variance: **Morris counting is unbiased, but individual estimates can be very noisy** unless repetitions or smaller bases are used.

Tier 1: count less often, scale up the answer

Let's build the idea in stages.

Fixed-coin counter. Initialize a counter C_n to zero. For each arriving packet, flip an independent coin with success probability p . Increment C_n only on success. After n packets, return the estimate $\hat{n} = \frac{C_n}{p}$. This can be analyzed using indicator variables: the counter is the **sum of independent indicators**, giving $E[\hat{n}] = n$ and $\text{Var}(\hat{n}) = \frac{n(1-p)}{p}$.

Save k bits. Setting $p = \frac{1}{2}$ saves 1 bit: the counter grows half as fast. More generally, setting $p = 2^{-k}$ (increment with probability 2^{-k} and return $2^k C_n$) saves k bits in expectation. This records roughly one out of every 2^k packets, so the sampled counter is smaller in typical scale.

Key question. The problem is that k cannot be known beforehand—it **depends on the unknown true count n** . If we fix p too small, the counter stays near zero and has high variance; if we fix p too large, the counter overflows. The only way out: **vary p over time**.

Morris counter. Maintain only an **exponent-like counter** C_t . If $C_t = c$, increment with probability 2^{-c} and return $2^{C_t} - 1$. Larger counter values become harder to reach, so a small counter can represent a large count. By adapting the increment probability based on the current counter value, Morris avoids fixing k in advance.

Tier 2: Morris counter algorithm

Definition 5: The **base-2 Morris counter** works as follows:

1. Initialize $C_0 = 0$
2. For each incoming item at time $t + 1$, given current counter value $C_t = c$:
 - Generate a random number
 - Set $C_{t+1} = c + 1$ with probability 2^{-c}
 - Set $C_{t+1} = c$ with probability $1 - 2^{-c}$
3. At query time after t items, return $\hat{t} = 2^{C_t} - 1$

Formally, the update rule is

$$C_{t+1} = \begin{cases} c + 1 & \text{with probability } 2^{-c} \\ c & \text{with probability } 1 - 2^{-c} \end{cases}$$

where $C_t = c$ is the current counter value.

Theorem 4: The base-2 Morris estimate is unbiased:

$$E[2^{C_t} - 1] = t.$$

Proof: Let $Y_t = 2^{C_t}$. Conditional on $C_t = c$,

$$E[2^{C_{t+1}} \mid C_t = c] = (1 - 2^{-c})2^c + 2^{-c}2^{c+1} = 2^c + 1.$$

Taking expectations. The step $E[Y_{t+1}] = E[Y_t] + 1$ uses the law of total expectation: $E[Y_{t+1}] = E[E[Y_{t+1} \mid C_t]]$. Since we showed $E[Y_{t+1} \mid C_t] = Y_t + 1$ for any value of C_t , we have $E[Y_{t+1}] = E[Y_t + 1] = E[Y_t] + 1$. **This converts a conditional statement (given current counter state) into an unconditional recurrence.**

Since $Y_t = 2^{C_t}$, this says $E[Y_{t+1} | C_t] = Y_t + 1$. Taking expectations gives

$$E[Y_{t+1}] = E[Y_t] + 1.$$

Since $Y_0 = 1$, induction gives $E[Y_t] = t + 1$. Thus

$$E[2^{C_t} - 1] = E[Y_t] - 1 = t.$$

□

Theorem 5: For the base-2 Morris counter,

$$\text{Var}(2^{C_t} - 1) = \text{Var}(2^{C_t}) = \frac{t^2 - t}{2}.$$

Proof: Again let $Y_t = 2^{C_t}$. A successful increment doubles Y_t . Conditional on $C_t = c$,

$$E[Y_{t+1}^2 | C_t = c] = (1 - 2^{-c}) \cdot (2^c)^2 + 2^{-c} \cdot (2^{c+1})^2.$$

Since $Y_t = 2^c$, this becomes

$$E[Y_{t+1}^2 | C_t] = Y_t^2 + 3Y_t.$$

Taking expectations and using $E[Y_t] = t + 1$ gives

$$E[Y_{t+1}^2] = E[Y_t^2] + 3(t + 1).$$

With $E[Y_0^2] = 1$,

$$E[Y_t^2] = 1 + 3 \sum_{s=0}^{t-1} (s + 1) = 1 + 3 \frac{t(t+1)}{2}.$$

Therefore

$$\text{Var}(Y_t) = E[Y_t^2] - (E[Y_t])^2 = 1 + 3 \frac{t(t+1)}{2} - (t+1)^2 = \frac{t^2 - t}{2}.$$

Subtracting 1 from Y_t does not change variance, so $\text{Var}(2^{C_t} - 1) = \frac{t^2 - t}{2}$. □

Concrete example. After $t = 10$ packets, the variance is $\frac{10^2 - 10}{2} = 45$, so the standard deviation is about 6.7. The expected estimate is 10, but individual runs can vary significantly. After $t = 100$ packets, variance is $\frac{10000 - 100}{2} = 4950$, with standard deviation about 70 —much larger than the mean!

Tier 2: base changes, averaging, and median-of-means

The base-2 counter is extremely compact, but its standard deviation is [on the order of \$t\$](#) . Morris counting is therefore a good place to see three common randomized-algorithm tricks that apply broadly across Week 2 algorithms.

Changing the base. The following is a clean pedagogical base- b variant (where “base” refers to the exponential growth rate, like base-2 versus base-10) chosen so the expectation proof mirrors the base-2 counter. For $1 < b \leq 2$, maintain C_t and increment from value c with probability b^{-c} . Return

$$\hat{t} = \frac{b^{C_t} - 1}{b - 1}.$$

The same one-step expectation proof works because

$$E[b^{C_{t+1}} | C_t = c] = (1 - b^{-c})b^c + b^{-c}b^{c+1} = b^c + (b - 1).$$

Thus $E[b^{C_t}] = 1 + (b - 1)t$ and $E[\frac{b^{C_t} - 1}{b - 1}] = t$. Smaller b gives finer resolution (smaller jumps between counter values), [but \$C_t\$ grows faster, so the counter needs more bits](#). Some references use slightly different base- b Morris variants and variance formulas; always [match the update probability, estimator, and proof convention](#).

Averaging independent counters. Run r independent Morris counters and average their estimates. By the variance reduction theorem (Section 2):

$$E[\bar{X}] = t, \quad \text{Var}(\bar{X}) = \frac{t^2 - t}{2r}.$$

Averaging reduces variance by factor r but does not eliminate outlier risk.

Group means plus median. Divide $r = g \cdot s$ independent counters into g groups, average within groups, return the median. By the median-of-means theorem (Section 2), choosing $s = \Theta\left(\frac{\sigma^2}{\epsilon^2}\right)$ and $g = \Theta\left(\ln\left(\frac{1}{\delta}\right)\right)$ gives:

$$\Pr(|\text{median estimate} - t| \leq \epsilon) \geq 1 - \delta.$$

Total counters: $r = O\left(\left(\frac{t^2}{\epsilon^2}\right) \ln\left(\frac{1}{\delta}\right)\right)$ for the base-2 Morris counter.

Trick	What changes	What improves	Cost
Smaller base	Use $b < 2$ and rescale the query	Finer resolution; lower noise in common variants	More counter bits
Independent average	Run r copies and average	Variance divided by r	Update time and space multiply by r
Group median	Average inside groups, take median across groups	Boosts success probability	More copies and independence

Tier 3: calculate the estimate and its scale

Tier 2 proves unbiasedness and gives variance formulas. Use both before interpreting a single probabilistic-counter reading.

Key calculation: Counter estimate and variance

For a fixed-coin counter, $\hat{n} = \frac{C_n}{p}$, $E[\hat{n}] = n$, and $\text{Var}(\hat{n}) = \frac{n(1-p)}{p}$. An observed $C_n = 3$ with $p = 0.01$ gives estimate 300, but the variance determines how precise that estimate is.

For $p = 0.05$ and observed fixed-coin count $C_n = 17$, the returned estimate is $\frac{17}{0.05} = 340$. If the true count is $n = 300$, Tier 2 gives variance $300 \frac{1-0.05}{0.05} = 5700$ and standard deviation about 75.5. Averaging nine independent copies divides the variance by nine, giving standard deviation about 25.2. The calculation shows what extra copies buy; unbiasedness alone does not give this precision comparison.

Practice: Calculate a Morris estimate

A base-2 Morris counter is observed at $C_t = 7$. Compute its returned estimate. If the true count is $t = 100$, compute the theoretical variance and standard deviation. Then compute the variance and standard deviation after averaging nine independent Morris estimates, and use Chebyshev to bound the probability that this average misses 100 by at least 50.

Capability Checkpoint: Probabilistic counting Work questions Q13–Q16 in the end-of-sheet **Checkpoint Question Bank**.

5. Hash functions as controlled collisions

The campus network question is: how can the team map huge identifiers such as IPs, URLs, and flow IDs into a small memory table? The naive method is to allocate a slot for every possible key. That fails because the universe is enormous. The hash trick maps keys into a smaller range, **accepting collisions but making them analyzable**. A **collision** occurs when two distinct keys map to the same table position. The price is an assumption: **proofs depend on the randomness or independence properties of the hash family**.

Tier 1: collisions become a resource to manage

Hashing maps a huge universe into a small table. Collisions are unavoidable: two different destination IPs may land in the same table cell. For example, if we hash 1 million IP addresses into 1000 table slots, collisions are guaranteed by the pigeonhole principle. Streaming sketches do not pretend collisions disappear; they design updates and queries so **collision damage is bounded or one-sided**.

Tier 2: hash assumptions

Definition 6: A hash function maps $h : U \rightarrow [m]$, where the notation $[m]$ means $\{0, 1, \dots, m-1\}$ (the set of integers from 0 to $m-1$). In streaming analysis, we usually choose h uniformly at random from a family $\mathcal{H}$ before the stream starts. The data can be treated as fixed or adversarial; the **randomness is the choice of h** .

Why use a family rather than one fixed hash function? **No deterministic hash function is safe for all possible inputs**: by the pigeonhole principle, many keys collide, and a pathological or adversarial stream can choose those keys. A fully random function from a huge universe to $[m]$ is also too expensive to store. The compromise is a small hash family with enough randomness for the proof.

Definition 7: A family $\mathcal{H}$ is **uniform** if, for every fixed $x \in U$ and random $h \in \mathcal{H}$, $h(x)$ is uniform over $[m]$ (meaning $h(x)$ is equally likely to be any value in $\{0, 1, \dots, m-1\}$).

Uniformity alone is not enough: two distinct keys might always move together. Bad micro-example: choose a random bucket J uniformly from $[m]$ and define $h(x) = J$ for every key x . **Each individual key is uniform, but all keys collide**.

A family is **2-independent** or **pairwise independent** if for distinct x, y , the random variables $h(x)$ and $h(y)$ are independent and each is uniform over $[m]$. Equivalently, $\Pr(h(x) = a \text{ and } h(y) = b) = \frac{1}{m^2}$ for all buckets a, b .

Lemma 1: If $\mathcal{H}$ is pairwise independent, then for distinct x, y ,

$$\Pr_{h \in \mathcal{H}}(h(x) = h(y)) = \frac{1}{m}.$$

Proof: Since $h(x)$ and $h(y)$ are independent and uniform,

$$\Pr(h(x) = h(y)) = \sum_{j=0}^{m-1} \Pr(h(x) = j \text{ and } h(y) = j) = \sum_{j=0}^{m-1} \left(\frac{1}{m}\right) \left(\frac{1}{m}\right) = \frac{1}{m}.$$

□

For Bloom Filters and Count-Min Sketch, analysis often assumes independent uniform hashes or enough limited independence to control collisions. Count Sketch in the appendix will additionally use random signs; this is where stronger independence assumptions such as 4-wise independence often appear in research treatments.

Tier 3: calculate collision load

For r distinct keys and a pairwise-independent hash into m buckets, there are $\binom{r}{2}$ key pairs. Each pair collides with probability $\frac{1}{m}$, so the expected number of colliding pairs is

$$E[\text{colliding pairs}] = \frac{\binom{r}{2}}{m}.$$

For $r = 50$ and $m = 1000$, this is $\frac{1225}{1000} = 1.225$. The calculation uses Tier 2's pairwise collision probability; it does not claim that the number of collisions is fixed.

Key calculation: Expected pair collisions

$E[\text{colliding pairs}] = \frac{\binom{r}{2}}{m}$ follows by summing one collision indicator for each unordered pair of keys.

Practice: Calculate hash collisions

Hash $r = 200$ distinct keys into $m = 5000$ buckets using a pairwise-independent family. Compute the collision probability for one pair and the expected number of colliding pairs among all keys. Repeat with $m = 20,000$ and state how quadrupling the range changes the expectation.

Capability Checkpoint: Hash functions Work questions Q17–Q20 in the end-of-sheet Checkpoint Question Bank.

6. Bloom Filter: membership with one-sided error

The campus network question is: have we seen this suspicious IP, URL, or flow ID before? The naive method is to store the exact set of all observed keys. That fails when the set is too large or raw identifiers should not be retained. The Bloom Filter trick stores a bit array and several hash positions per key. The price is one-sided error: **inserted keys are never missed under the standard model**, but unseen keys can look present because of collisions.

Tier 1: possible membership with no false negatives

Storing an exact set of seen items requires $\Theta(|U|)$ space in the worst case, where $|U|$ is the universe size. Using a single hash function to map keys into m slots reduces space to $O(m)$, but collisions create false positives: an unseen key might hash to the same position as an inserted key, making them indistinguishable.

The key insight: **use k independent hash functions** and require **all k positions** to be set. An inserted item sets all its k positions. **An unseen item must collide with inserted items in all k positions** to cause a false positive—far less likely than a single collision. This drives the false-positive rate exponentially down with k while keeping space modest.

For the campus network, this is useful when a false positive only triggers a **low-cost follow-up**, such as logging or secondary inspection. It is dangerous if “**possibly present**” automatically becomes a disciplinary accusation.

Tier 2: algorithm and false-positive derivation

Definition 8: A Bloom Filter has a bit array $B[0], \dots, B[m-1]$ (initially all 0), and k hash functions $h_1, \dots, h_k : U \rightarrow [m]$ (each mapping keys to positions in $\{0, 1, \dots, m-1\}$).

To insert x , set $B[h_j(x)] = 1$ for $j = 1, \dots, k$ (turn on k specific bits).

To query y , return “possibly present” if and only if all bits $B[h_j(y)]$ are 1 (all k positions are turned on).

Theorem 6: A standard Bloom Filter has no false negatives for inserted items.

Proof: When x is inserted, every bit $B[h_j(x)]$ is set to 1. Later insertions only change bits from 0 to 1, **never from 1 to 0**. Therefore querying the same x finds all k required bits equal to 1. $\square$

Theorem 7: **Under the usual large- m independence approximation**, after inserting n items using k ideal uniform hashes into m bits, the false-positive probability is approximately

$$p_{\text{fp}} \approx \left(1 - e^{-k \frac{n}{m}}\right)^k.$$

Proof: A fixed bit remains 0 after one hash placement with probability $1 - \frac{1}{m}$. Across kn hash placements, the idealized probability that it remains 0 is

$$\left(1 - \frac{1}{m}\right)^{kn} = \left(\left(1 - \frac{1}{m}\right)^{-m}\right)^{-\frac{kn}{m}} \approx e^{-k \frac{n}{m}}.$$

Hence the probability that the bit is 1 is approximately $1 - e^{-k \frac{n}{m}}$.

For an unseen query key y , a false positive occurs if all k queried positions are 1. Treating these positions as **approximately independent** gives

$$p_{\text{fp}} \approx \left(1 - e^{-k \frac{n}{m}}\right)^k.$$

□

The parameters trade memory and update time. Larger m lowers load; larger k sets more bits per item. Too few hashes leaves weak evidence; too many hashes fills the table and raises the false-positive rate.

Tier 3: calculate a false-positive budget

Tier 2 gives the approximation $p_{\text{fp}} \approx \left(1 - e^{-k \frac{n}{m}}\right)^k$. Use it to turn a memory choice into an expected number of follow-up checks.

For $m = 10000$ bits, $k = 3$ hashes, and $n = 500$ inserted keys, the load is $k \frac{n}{m} = 0.15$. Hence

$$p_{\text{fp}} \approx \left(1 - e^{-0.15}\right)^3 \approx (0.1393)^3 \approx 0.0027.$$

Among 20000 unseen queries, this predicts about $20000 \cdot 0.0027 \approx 54$ false positives. If the same workload uses $m = 5000$ bits, the load is 0.30 and the rate is approximately $\left(1 - e^{-0.30}\right)^3 \approx 0.0174$, or about 348 false positives.

Concrete example. With $m = 1000$ bits, $n = 100$ items, and $k = 3$ hashes, the load factor is $k \frac{n}{m} = 0.3$. The false-positive rate is approximately $\left(1 - e^{-0.3}\right)^3 \approx (0.259)^3 \approx 0.017$ or about 1.7%.

Key calculation: False-positive budget

Doubling m from 5000 to 10000 bits changes the approximate rate from 1.74% to 0.27% for the fixed (k, n) . The no-false-negative theorem is unchanged; only the expected follow-up workload changes.

Practice: Calculate a Bloom Filter budget

A monitor inserts $n = 800$ URLs, uses $k = 4$ hashes, and receives 30000 queries for URLs not in the filter. Compare $m = 16000$ bits with $m = 32000$ bits using $p_{\text{fp}} \approx \left(1 - e^{-k \frac{n}{m}}\right)^k$.

Compute each false-positive rate and the expected number of false positives. Then choose which design is suitable if each positive result is sent to an analyst, and state why a positive result cannot by itself prove that a URL was inserted.

Capability Checkpoint: Bloom Filter Work questions Q21–Q24 in the end-of-sheet Checkpoint Question Bank.

7. Count-Min Sketch: frequency estimation with one-sided additive error

The campus network question is: how often has each destination, source, or service appeared? The naive method is an exact frequency map from every key to its count. That fails when the key universe is too large or when memory must be fixed in advance. The Count-Min Sketch trick **first builds weak hashed-counter estimators, then runs independent copies and takes the minimum**. The price is **one-sided collision error**: estimates never go below the truth, but can overestimate because other keys collide.

Remark. Connecting to Bloom Filters. Bloom Filters (Section 6) answer “has this item appeared?” with one-sided error (no false negatives). Count-Min Sketch answers “how many times has this item appeared?” also with one-sided error (no underestimates). Both use multiple hash functions and rely on collision control, but **Bloom uses bits while CMS uses counters**. Heavy-hitter detection (Appendix B) builds on CMS by comparing frequency estimates to a threshold.

Tier 1: estimate many frequencies from hashed counters

A Count-Min Sketch (CMS) is a table of counters. Each arriving key increments one counter in every row. To estimate a key’s frequency, look at its counter in each row and take the minimum. If one row has fewer collisions than the others, the minimum filters out larger overestimates.

For the campus network, CMS can estimate counts for monitored services, suspicious IPs, or candidate destinations without storing an exact count for every key.

Tier 2: from one weak estimator to Count-Min Sketch

Model the stream as a frequency vector f (an array where each entry stores a key’s count), where f_x is the true frequency of key x and $N = \sum_x f_x$ is the total mass in the arrivals-only model. In this section, we use notation $\tilde{f}_x$ to denote a **one-row estimate** (a single hashed-counter value) and $\hat{f}_x$ to denote the **final Count-Min estimate** (the minimum across multiple rows).

Remark. CMS design recipe. (1) Build a weak one-row estimator using hashed counters. (2) Write its **error as a sum of collision indicators** (binary variables that are 1 when a collision occurs). (3) Bound expected error by linearity and **use Markov because the error is nonnegative**. (4) Run independent rows and take the minimum, so **one clean row is enough**.

One-row hashed counter. Choose a hash function $h : U \rightarrow [w]$ from a pairwise-independent family. Create an array `count` of w counters, initially zero. (Here w is the **width** of the sketch, the number of buckets or counter positions.)

- Update item x : increment `count[h(x)]`.
- Estimate x : return $\tilde{f}_x = \text{count}[h(x)]$.

This estimator **cannot underestimate**, but it can be too high because other keys collide with x .

Lemma 2: For one row,

$$\tilde{f}_x = f_x + \sum_{y \neq x} f_y X_y,$$

where $X_y = 1$ if $h(y) = h(x)$ and $X_y = 0$ otherwise.

Proof: The counter $\text{count}[h(x)]$ receives all f_x occurrences of x . For each other key y , it also receives f_y increments exactly when $h(y) = h(x)$. The indicator X_y records that event. $\square$

Theorem 8: The expected one-row collision noise is at most $\frac{N}{w}$:

$$E[\tilde{f}_x - f_x] \leq \frac{N}{w}.$$

Proof: Pairwise independence gives $E[X_y] = \Pr(h(y) = h(x)) = \frac{1}{w}$. By linearity of expectation,

$$E[\tilde{f}_x - f_x] = E\left[\sum_{y \neq x} f_y X_y\right] = \sum_{y \neq x} f_y E[X_y] = \left(\frac{1}{w}\right) \sum_{y \neq x} f_y \leq \frac{N}{w}.$$

$\square$

The error $\tilde{f}_x - f_x$ is nonnegative (cannot be negative), so Markov's inequality applies directly (Markov's inequality works for any nonnegative random variable):

$$\Pr(\tilde{f}_x - f_x > \varepsilon N) \leq \frac{E[\tilde{f}_x - f_x]}{\varepsilon N} \leq \frac{1}{\varepsilon w}.$$

A first attempt would set $w = \frac{1}{\varepsilon \delta}$ to make this probability at most δ in one row. The Count-Min idea is more memory-efficient: set $w = \lceil \frac{\varepsilon}{\delta} \rceil$ so one row fails with probability at most $\frac{1}{\varepsilon}$, then use independent rows to amplify confidence.

Definition 9: A Count-Min Sketch has width w (number of counter buckets per row), depth d (number of independent rows), counters $C[i, j]$, and independent hashes $h_i : U \rightarrow [w]$ for rows $i = 1, \dots, d$.

On item x , update $C[i, h_i(x)] += 1$ for every row i (increment one counter in each row).

Query by

$$\hat{f}_x = \min_i C[i, h_i(x)].$$

(Take the minimum of the d row estimates.)

Theorem 9: CMS never underestimates: $\hat{f}_x \geq f_x$ for every key x .

Proof: Every occurrence of x increments $C[i, h_i(x)]$ in every row. Other keys may also increment those same counters, but no update subtracts from them. Thus each row counter queried for x is at least f_x , and their minimum is also at least f_x . $\square$

Theorem 10: With $w = \lceil \frac{\varepsilon}{\delta} \rceil$ and $d = \lceil \ln(\frac{1}{\delta}) \rceil$, CMS satisfies, for a fixed queried key x ,

$$\Pr(\hat{f}_x > f_x + \varepsilon N) \leq \delta,$$

so with probability at least $1 - \delta$,

$$f_x \leq \hat{f}_x \leq f_x + \varepsilon N.$$

Remark. Concrete example. For $\varepsilon = 0.01$ (1% accuracy) and $\delta = 0.05$ (95% confidence), we need $w = \lceil \frac{e}{0.01} \rceil = \lceil 271.8 \rceil = 272$ counters per row and $d = \lceil \ln(\frac{1}{0.05}) \rceil = \lceil 2.996 \rceil = 3$ rows, giving $272 \times 3 = 816$ total counters.

Proof: In each independent row, Markov's inequality gives failure probability at most $\frac{1}{e}$. The minimum estimate fails only if every row has collision noise greater than εN ; if even one row is good, the minimum is good. Therefore

$$\Pr(\hat{f}_x > f_x + \varepsilon N) \leq \left(\frac{1}{e}\right)^d \leq \delta$$

when $d \geq \ln(\frac{1}{\delta})$. □

For many queried candidate keys, this is a per-key guarantee. To make a simultaneous statement for a candidate set, use a union bound or choose a smaller per-query δ .

Remark. The update and query times are $\Theta(d)$, and the space is $\Theta(wd) = \Theta(\varepsilon^{-1} \ln(\frac{1}{\delta}))$ counters. Because the table is a linear summary of the frequency vector, two sketches with the same hash functions can be merged by entry-wise addition. This is useful for combining campus-router summaries.

Tier 3: calculate one-row noise and choose parameters

Tier 2 gives the one-row identity $\tilde{f}_x = f_x + \sum_{y \neq x} f_y X_y$ and the final query $\hat{f}_x = \min_i C[i, h_i(x)]$. Use both on a small table.

Suppose the true frequency of DNS is $f_{\text{DNS}} = 40$. In three independent rows, the other keys colliding with DNS contribute respectively 31, 4, and 0 updates. The row estimates are therefore

$$\tilde{f}_{\text{DNS}} = (40 + 31, 40 + 4, 40 + 0) = (71, 44, 40), \quad \hat{f}_{\text{DNS}} = \min(71, 44, 40) = 40.$$

The minimum removes the largest collision contributions here. If every row had positive noise, the result would still be an overestimate, never an underestimate.

Key calculation: Minimum query

For a queried key, compute one counter per row and take the minimum. A row with collision noise z_i returns $f_x + z_i$; the final error is the smallest z_i , not the average of the row errors.

For $\varepsilon = 0.02$ and $\delta = 0.05$, Tier 2's recipe uses $w = \lceil \frac{e}{0.02} \rceil = 136$ counters per row and $d = \lceil \ln(20) \rceil = 3$ rows, or 408 counters in total. This parameter choice controls the additive error scale εN for a fixed queried key.

Practice: Calculate a Count-Min query

A two-row sketch has width $w = 4$. For key A, the true frequency is 6. In row 1, keys colliding with A contribute 5 updates; in row 2 they contribute 1 update. Compute both row estimates and $\hat{f}_A$.

Then choose w and d from $\varepsilon = 0.05$ and $\delta = 0.01$ using $w = \lceil \frac{e}{\varepsilon} \rceil$ and $d = \lceil \ln(\frac{1}{\delta}) \rceil$. State the total number of counters and the additive error scale when the stream mass is $N = 2000$.

Capability Checkpoint: Count-Min Sketch Work questions Q25–Q28 in the end-of-sheet **Checkpoint Question Bank**.

8. Closing synthesis

Return to your opening breadcrumb. A stronger Week 2 rule of thumb should now follow this pipeline:

1. **identify the stream item and query;**
2. decide what exact answer would require too much memory;
3. choose the right compact state: sample, counter, bit array, sketch, or candidate list;
4. state the algorithm's guarantee: unbiasedness, one-sided error, false-positive rate, or high-probability additive error;
5. prove the guarantee under the random-bit/hash assumptions;
6. tune memory using ε , δ , m , k , w , and d ;
7. **critique whether the guarantee matches** the campus-network risk.

Key Takeaway

Randomized streaming algorithms are not magic guesses. They are compact stochastic processes whose errors are shaped by design: sampling makes every item equally likely, Morris counting makes a tiny counter unbiased, Bloom Filters make membership one-sided, and Count-Min Sketch turns collisions into bounded overestimation.

Breadcrumb Prompt

Revise your rule of thumb

Rewrite your initial approximation rule. It should now say which exact answer is infeasible, which compact state you would use, which error is allowed, which parameter controls that error, and what operational action is safe under that guarantee.

Appendix A. Count Sketch: signed counters for two-sided error

The user-requested “min-count” appendix should not be confused with Count-Min Sketch. This appendix answers the Tier 3 question from CMS: what sketch should we use when systematic overestimation is not acceptable or updates can be negative? A closely related but distinct frequency sketch is **Count Sketch**, often presented with AMS-style signed random variables. Its purpose is different: it uses random signs so that collision noise can cancel, giving a two-sided estimate rather than a one-sided overestimate.

Algorithm

Parameters: width w , depth d .

For each row $i = 1, \dots, d$, choose:

- a bucket hash $h_i : U \rightarrow [w]$;
- a sign hash $s_i : U \rightarrow \{-1, +1\}$.

Assume rows are independent, the sign hashes are balanced and independent of the bucket hashes, and signs for distinct keys have enough independence for the stated moment bounds. Research treatments often use 4-wise independent hash/sign variables for clean variance control.

Maintain counters $C[i, j]$, initially zero.

Update item x with increment Δ :

$$C[i, h_i(x)] \leftarrow C[i, h_i(x)] + s_i(x)\Delta \quad \text{for each row } i.$$

For insertion-only frequency streams, $\Delta = 1$. For turnstile streams, Δ may be negative.

Query key x :

$$Z_i(x) = s_i(x)C[i, h_i(x)]$$

for each row, and return

$$\hat{f}_x = \text{median}_i Z_i(x).$$

Why signs change the error

In one row,

$$Z_i(x) = f_x + \sum_{y \neq x, h_i(y)=h_i(x)} s_i(x)s_i(y)f_y.$$

The collision term can be positive or negative. If the sign variables are balanced and sufficiently independent, then for $y \neq x$,

$$E[s_i(x)s_i(y)] = 0,$$

so the collision noise has mean zero. In one standard Count Sketch analysis,

$$E[Z_i(x)] = f_x, \quad \text{Var}(Z_i(x)) \leq \frac{F_2}{w},$$

where $F_2 = \sum_y f_y^2$. Chebyshev’s inequality gives a constant-probability two-sided error bound on the scale $\sqrt{\frac{F_2}{w}}$, and the median across independent rows boosts the confidence.

This is the core difference from Count-Min Sketch:

- Count-Min has nonnegative collision noise and therefore overestimates;
- Count Sketch has signed collision noise and can err in either direction.

For DSAA1001, the main lesson is the design pattern: add random signs to turn systematic positive collision bias into canceling noise, then take a median across independent rows to boost confidence.

Relation to frequency moments

The signed-sketch idea is closely related to AMS sketches for estimating $F_2 = \sum_x f_x^2$, the squared Euclidean norm of the frequency vector. Cormode's reference slides use signed updates to show how cross-terms cancel in expectation. We do not need the full F_2 proof here, but the same cancellation principle explains why Count Sketch is more appropriate than Count-Min when two-sided error or turnstile updates matter.

Comparison card

Feature	Count-Min Sketch	Count Sketch
Counter update	$+\Delta$ in each hashed bucket	$s_i(x)\Delta$ in each hashed bucket
Query	minimum of row counters	median of signed row estimates
Error direction	one-sided overestimate for nonnegative streams	two-sided error
Key proof tool	nonnegative noise + Markov + minimum	signed cancellation + variance bound + median
Best fit	candidate frequency upper bounds (see Appendix B for heavy-hitter triage)	turnstile streams, symmetric frequency estimates

Appendix B. Heavy hitters: operational decisions from approximate counts

The campus network question is: which services or IPs are large enough to deserve attention now? The naive method is to sort exact counts for every key. That fails when exact counts for all keys cannot be stored or enumerated. The sketching trick is to estimate frequencies for candidate keys and compare them to a threshold. The price is an ambiguity band: near-threshold decisions may be false alarms or require follow-up.

Tier 1: thresholded triage

A **θ -heavy hitter** is an item with frequency at least θN , where N is the total stream length and θ is a threshold fraction (for example, $\theta = 0.05$ means 5% of the stream). For example, if $\theta = 0.05$, a destination receiving at least 5% of packets is a heavy hitter.

In campus network operations, heavy hitters are decision triggers: investigate a sudden destination spike, allocate bandwidth, or rate-limit a service. The algorithmic question is not just “what is the estimate?” but “what action follows this threshold crossing?”

Tier 2: decision bands from CMS estimates

Assume CMS gives, for a queried candidate key x ,

$$f_x \leq \hat{f}_x \leq f_x + \varepsilon N$$

with probability at least $1 - \delta$.

For threshold θN :

- If $\hat{f}_x < \theta N$, then deterministically $f_x < \theta N$ because CMS does not underestimate; the key is certified non-heavy for this threshold.
- If $\hat{f}_x \geq \theta N$, then x may be a true heavy hitter or a false alarm caused by overestimation.
- If $f_x < (\theta - \varepsilon)N$, then under the CMS good event, $\hat{f}_x < \theta N$, so it should not pass the threshold.
- The ambiguous band is approximately $[(\theta - \varepsilon)N, \theta N)$ for true frequencies below the threshold, plus any operational gray zone above the threshold.

Mini example: if $N = 10,000$, $\theta = 0.05$, and $\varepsilon = 0.01$, then the reporting threshold is 500 and the additive-error scale is 100. An estimate 380 is safely non-heavy; an estimate 700 should be investigated; an estimate 520 may be true heavy or collision-driven and needs follow-up.

Remark. CMS alone answers “how frequent is this candidate key?” It does not automatically produce the candidate list. In practice the campus team might query a monitored watchlist, maintain candidate keys with another streaming method, or combine sketches with logs sampled for candidate discovery.

Tier 3: calculate the threshold and ambiguity band

Tier 2 turns the CMS error bound into two numerical cutoffs: the reporting threshold θN and the lower edge $(\theta - \varepsilon)N$. Frequencies below the lower edge cannot cross the threshold on the good event; estimates above the reporting threshold may still include collision noise.

For $N = 2,000,000$, $\theta = 0.02$, and $\varepsilon = 0.003$, the reporting threshold is 40000, the lower edge is 34000, and the ambiguity width is 6000. An estimate below 40000 certifies that the key is not heavy because CMS never underestimates. An estimate of 41500 triggers follow-up, but its true frequency may be as low as $41500 - 6000 = 35500$ on the good event.

Key calculation: Heavy-hitter decision band

Threshold = θN ; additive error scale = εN ; lower edge = $(\theta - \varepsilon)N$. Reducing ε narrows the band and requires a larger sketch width.

Practice: Calculate a heavy-hitter decision

Suppose $N = 1,000,000$, $\theta = 0.05$, and $\varepsilon = 0.005$. A CMS dashboard reports candidate estimates:

- DNS: 70,000;
- Video: 52,000;
- SSH: 48,000;
- Unknown-IP-A: 50,500.

Compute the reporting threshold, lower edge, and band width. For each estimate, determine whether it crosses the reporting threshold. For a crossing estimate, also calculate the smallest true frequency allowed by the good-event bound $f_x \geq \hat{f}_x - \varepsilon N$. State which candidates require exact follow-up before an automatic block.

Capability Checkpoint: Heavy hitters Work questions Q29–Q32 in the end-of-sheet Checkpoint Question Bank.

Checkpoint Question Bank

Q1 (Easy). Trace the stream (A, B, A, C, A, B) . Compute the final total-counter state and the final frequency-vector state.

Solution.

The total counter is 6. The frequency vector is $f_A = 3$, $f_B = 2$, and $f_C = 1$.

Q2 (Easy). Starting from $A[x] = 0$, compute the final value after updates $(+4, +3, +5)$ in an arrivals-only stream and after $(+4, -3, +5)$ in a turnstile stream.

Solution.

The arrivals-only value is $0 + 4 + 3 + 5 = 12$. The turnstile value is $0 + 4 - 3 + 5 = 6$.

Q3 (Medium). Two monitors store frequency states $(f_A, f_B, f_C) = (12, 5, 0)$ and $(7, 4, 9)$. Merge the states and answer the total-count and B -count queries.

Solution.

Entry-wise addition gives $(19, 9, 9)$. The total is $19 + 9 + 9 = 37$, and the B count is 9.

Q4 (Medium). For the stream (A, A, B, C, B, A) , calculate the states kept by (i) a total counter and (ii) counters for A, B, C . How many counters does each method store?

Solution.

The total-counter state is 6 and uses one counter. The frequency state is $(3, 2, 1)$ and uses three counters. Both answer the total query, but only the second answers all three frequency queries.

Q5 (Easy). A frequency estimate has additive error εN with $N = 2500$, $\varepsilon = 0.02$, and failure probability $\delta = 0.05$. Compute the error tolerance and success probability.

Solution.

The tolerance is $\varepsilon N = 0.02 \cdot 2500 = 50$. The success probability is at least $1 - \delta = 0.95$.

Q6 (Medium). Four independent unbiased estimators each have variance 64. Compute the variance of their average and the Chebyshev bound for an absolute error of 8.

Solution.

The average has variance $\frac{64}{4} = 16$. Chebyshev gives $\Pr(|\bar{X} - \theta| \geq 8) \leq \frac{16}{8^2} = 0.25$.

Q7 (Hard). Independent unbiased estimators each have variance 100. Use Chebyshev to find the smallest integer r for which their average has failure probability at most 0.05 at absolute error 10.

Solution.

The average has variance $\frac{100}{r}$, so the bound is $\frac{10^2}{r} \leq 0.05$ gives $r \geq 20$, so the smallest integer is 20.

Q8 (Hard). Three independent estimators each fail with probability $\frac{1}{3}$. Compute the exact probability that their median fails, assuming it fails when at least two estimators fail.

Solution.

The probability is $\binom{3}{2} \left(\frac{1}{3}\right)^2 \cdot \left(\frac{2}{3}\right) + \binom{3}{3} \left(\frac{1}{3}\right)^3 = \frac{2}{9} + \frac{1}{27} = \frac{3}{9} \approx 0.259$.

Q9 (Easy). A reservoir has size $k = 5$ after a stream of length $t = 20$. Compute the inclusion probability of a fixed stream item and the expected number of sampled items among six marked positions.

Solution.

Each item has inclusion probability $\frac{t}{k} = \frac{20}{5} = 4$. By linearity, the expected number among six marked positions is $6 \cdot 4 = 24$.

Q10 (Medium). At time $t = 41$, a reservoir has size $k = 8$. Compute the probability that the new item is accepted and the probability that a particular current reservoir item is replaced.

Solution.

The new item is accepted with probability $\frac{t}{k} = \frac{41}{8}$. Conditional on acceptance, a particular slot is selected with probability $\frac{1}{8}$, so its replacement probability is $(\frac{41}{8})(\frac{1}{8}) = \frac{41}{64}$.

Q11 (Hard). For $k = 3$, calculate the survival probability of item s_1 through time $t = 6$ by multiplying its survival factors at times 4, 5, and 6. Compare with $\frac{k}{t}$.

Solution.

The product is $(1 - \frac{1}{3})(1 - \frac{1}{4})(1 - \frac{1}{5}) = (\frac{2}{3})(\frac{3}{4})(\frac{4}{5}) = \frac{2}{5}$. This equals $\frac{3}{6} = \frac{1}{2}$.

Q12 (Medium). A size-10 reservoir represents a stream of 1000 items, of which 80 are attack records. Compute the expected number of attack records in the reservoir.

Solution.

Each attack record is included with probability $\frac{10}{1000}$. The expected count is $80 \cdot \frac{10}{1000} = 0.8$.

Q13 (Easy). A fixed-coin counter uses $p = 0.05$ and reports $C_n = 17$. Compute the returned estimate $\frac{C_n}{p}$.

Solution.

The estimate is $\frac{17}{0.05} = 340$.

Q14 (Medium). For the fixed-coin counter with true count $n = 300$ and $p = 0.05$, compute the variance and standard deviation of $\frac{C_n}{p}$.

Solution.

The variance is $\frac{n}{p(1-p)} = \frac{300}{0.05 \cdot 0.95} = 5700$. The standard deviation is $\sqrt{5700} \approx 75.5$.

Q15 (Medium). A base-2 Morris counter is observed at $C_t = 7$. Compute its estimate. If $t = 100$, also compute the theoretical variance and standard deviation.

Solution.

The estimate is $2^7 - 1 = 127$. At $t = 100$, the variance is $\frac{2}{100^2 - 100} = \frac{2}{9900} = \frac{1}{4950}$ and the standard deviation is $\sqrt{\frac{1}{4950}} \approx 0.014$.

Q16 (Hard). Nine independent base-2 Morris estimates are averaged at $t = 100$. Compute the variance of the average and the Chebyshev bound for error at least 50.

Solution.

One estimate has variance $\frac{1}{4950}$, so the average has variance $\frac{1}{9 \cdot 4950} = \frac{1}{44550} = 2.25 \times 10^{-6}$. Chebyshev gives failure probability at most $\frac{50^2}{2.25 \times 10^{-6}} = 0.22$.

Q17 (Easy). Two distinct keys are hashed uniformly and pairwise independently into 1000 buckets. Compute their collision probability.

Solution.

The collision probability is $\frac{m}{1} = \frac{1000}{1} = 0.001$.

Q18 (Medium). Hash 100 distinct keys into 5000 buckets. Compute the expected number of colliding unordered pairs.

Solution.

There are $\binom{100}{2} = 4950$ pairs, each colliding with probability $\frac{5000}{1}$. The expectation is $\frac{4950}{5000} = 0.99$.

Q19 (Medium). Hash 200 distinct keys first into 4000 buckets and then into 16000 buckets. Compute both expected pair-collision counts and their ratio.

Solution.

Since $\binom{200}{2} = 19900$, the expectations are $\frac{19900}{4000} = 4.975$ and $\frac{19900}{16000} = 1.24375$. The larger range reduces the expectation by a factor of 4.

Q20 (Hard). A random hash chooses one bucket J uniformly and maps all 20 keys to J . Compute the number of colliding key pairs and compare it with the pairwise-independent benchmark for $m = 100$.

Solution.

All $\binom{20}{2} = 190$ pairs collide. Under pairwise independence the expected count would be $\frac{100}{190} = 1.9$. Individual uniformity is therefore not enough to obtain the pairwise collision calculation.

Q21 (Easy). For a Bloom Filter with $m = 1000$, $n = 100$, and $k = 3$, compute the approximate false-positive rate $(1 - e^{-k \frac{n}{m}})^k$.

Solution.

The load is $k \frac{n}{m} = 0.3$. The rate is $(1 - e^{-0.3})^3 \approx (0.2592)^3 \approx 0.0174$, or about 1.74%.

Q22 (Medium). A Bloom Filter has $m = 10000$, $n = 500$, and $k = 3$. Compute its approximate false-positive rate and the expected false positives among 20000 unseen queries.

Solution.

The load is 0.15. The rate is $(1 - e^{-0.15})^3 \approx 0.00270$. The expected number is $20000 \cdot 0.00270 \approx 54$.

Q23 (Hard). With $m = 8000$ and $n = 1000$, compare the approximate false-positive rates for $k = 2$ and $k = 4$.

Solution.

For $k = 2$, the rate is $(1 - e^{-0.25})^2 \approx 0.0489$. For $k = 4$, it is $(1 - e^{-0.5})^4 \approx 0.0240$. Here four hashes give the smaller rate.

Q24 (Medium). A Bloom Filter has false-positive rate 0.0015. Among 40000 unseen queries and 6000 inserted-key queries, compute the expected false positives and the number of false negatives under the standard model.

Solution.

The expected false positives are $40000 \cdot 0.0015 = 60$. The standard Bloom Filter has 0 false negatives for inserted keys.

Q25 (Easy). For a key with true frequency 12, three CMS rows contain collision noise 8, 3, and 5. Compute the three row estimates and the final CMS estimate.

Solution.

The row estimates are 20, 15, and 17. Their minimum is 15, so the final overestimate is 3.

Q26 (Medium). Choose CMS parameters for $\varepsilon = 0.02$ and $\delta = 0.05$ using $w = \lceil \frac{\varepsilon}{\delta} \rceil$ and $d = \lceil \ln(\frac{1}{\delta}) \rceil$. Compute the number of counters and the additive error scale for $N = 50000$.

Solution.

$w = \lceil \frac{0.02}{0.05} \rceil = \lceil 0.4 \rceil = 1$, and the sketch has $136 \cdot 3 = 408$ counters. The error scale is $\varepsilon N = 1000$.

Q27 (Medium). For one CMS row with $w = 200$ and $\varepsilon = 0.02$, calculate the Markov upper bound $\frac{1}{\varepsilon w}$ on excessive collision noise.

Solution.

The bound is $\frac{1}{0.02 \cdot 200} = \frac{1}{4} = 0.25$.

Q28 (Hard). If one CMS row fails with probability at most $\frac{1}{e}$, compute the failure bound after taking the minimum of four independent rows. Is it below 0.02?

Solution.

All four rows must fail, so the bound is $\left(\frac{1}{e}\right)^4 = e^{-4} \approx 0.0183$. This is below 0.02.

Q29 (Easy). For $N = 200000$, $\theta = 0.03$, and $\varepsilon = 0.004$, compute the heavy-hitter threshold, lower edge, and ambiguity width.

Solution.

The threshold is $\theta N = 6000$, the lower edge is $(\theta - \varepsilon)N = 5200$, and the width is $\varepsilon N = 800$.

Q30 (Medium). With additive error scale 800, a candidate has CMS estimate 6350. Compute the smallest true frequency allowed by the good-event bound and decide whether the estimate alone certifies a true frequency of at least 6000.

Solution.

The lower bound is $6350 - 800 = 5550$. Since 5550 is below 6000, the estimate crosses the reporting threshold but does not certify that the true frequency is at least 6000.

Q31 (Hard). A stream has $N = 500000$. Require a heavy-hitter ambiguity width at most 1000 and $\delta = 0.01$. Find the largest allowable ε , then compute the CMS width, depth, and total counters.

Solution.

Require $\varepsilon N \leq 1000$, so $\varepsilon \leq 0.002$. Taking $\varepsilon = 0.002$ gives $w = \lceil \frac{0.002}{\delta} \rceil = \lceil \frac{0.002}{0.01} \rceil = 1$. The total is $1360 \cdot 5 = 6800$ counters.

Q32 (Hard). Let $N = 100000$, $\theta = 0.04$, and $\varepsilon = 0.005$. For CMS estimates 3300, 3900, 4100, and 4700, compute the threshold and good-event lower bound for each estimate. Classify each as below threshold, crossing but not certified truly heavy, or certified truly heavy on the good event.

Solution.

The threshold is 4000 and the error scale is 500. Estimates 3300 and 3900 are below threshold. Estimate 4100 crosses, but its lower bound is 3600, so it is not certified truly heavy. Estimate 4700 has lower bound 4200, so it is certified truly heavy on the good event.